

Programming Challenges

1. Date Exceptions

Modify the `Date` class you wrote for Programming Challenge 1 of Chapter 13. The class should implement the following exception classes:

`InvalidDay` Throw when an invalid day (< 1 or > 31) is passed to the class.

`InvalidMonth` Throw when an invalid month (< 1 or > 12) is passed to the class.

Demonstrate the class in a driver program.

2. Time Format Exceptions

Modify the `milTime` class you created for Programming Challenge 4 of Chapter 15. The class should implement the following exceptions:

`BadHour` Throw when an invalid hour (< 0 or > 2359) is passed to the class.

`BadSeconds` Throw when an invalid number of seconds (< 0 or > 59) is passed to the class.

Demonstrate the class in a driver program.

3. Minimum/Maximum Templates

Write templates for the two functions `minimum` and `maximum`. The `minimum` function should accept two arguments and return the value of the argument that is the lesser of the two. The `maximum` function should accept two arguments and return the value of the argument that is the greater of the two. Design a simple driver program that demonstrates the templates with various data types.

4. Absolute Value Template

Write a function template that accepts an argument and returns its absolute value. The absolute value of a number is its value with no sign. For example, the absolute value of -5 is 5, and the absolute value of 2 is 2. Test the template in a simple driver program.

5. Total Template

Write a template for a function called `total`. The function should keep a running total of values entered by the user, then return the total. The argument sent into the function should be the number of values the function is to read. Test the template in a simple driver program that sends values of various types as arguments and displays the results.

6. `IntArray` Class Exception

Chapter 14 presented an `IntArray` class that dynamically creates an array of integers and performs bounds checking on the array. If an invalid subscript is used with the class, it displays an error message and aborts the program. Modify the class so it throws an exception instead.

7. `TestScores` Class

Write a class named `TestScores`. The class constructor should accept an array of test scores as its argument. The class should have a member function that returns the average of the test scores. If any test score in the array is negative or greater than 100, the class should throw an exception. Demonstrate the class in a program.

8. SimpleVector Modification

Modify the `SimpleVector` class template presented in this chapter to include the member functions `push_back` and `pop_back`. These functions should emulate the STL vector class member functions of the same name. (See Table 16-5.) The `push_back` function should accept an argument and insert its value at the end of the array. The `pop_back` function should accept no argument and remove the last element from the array. Test the class with a driver program.

9. SearchableVector Modification

Modify the `SearchableVector` class template presented in this chapter so that it performs a binary search instead of a linear search. Test the template in a driver program.

10. SortableVector Class Template

Write a class template named `SortableVector`. The class should be derived from the `SimpleVector` class presented in this chapter. It should have a member function that sorts the array elements in ascending order. (Use the sorting algorithm of your choice.) Test the template in a driver program.

11. Inheritance Modification

Assuming you have completed Programming Challenges 9 and 10, modify the inheritance hierarchy of the `SearchableVector` class template so it is derived from the `SortableVector` class instead of the `SimpleVector` class. Implement a member function named `sortAndSearch`, both a sort and a binary search.

12. Specialized Templates

In this chapter, the section *Specialized Templates* within Section 16.4 describes how to design templates that are specialized for one particular data type. The section introduces a method for specializing a version of the `SimpleVector` class template so it will work with strings. Complete the specialization for both the `SimpleVector` and `SearchableVector` templates. Demonstrate them with a simple driver program.

13. Rainfall vector

Modify Programming Challenge 2 in Chapter 7 (Rainfall Statistics) to use an STL vector instead of an array. Refer to the information in Tables 16-5 and 16-6 if you wish to use any of the member functions.

14. Test Scores vector

Modify Programming Challenge 2 in Chapter 9 (Test Scores #1) to use an STL vector instead of a dynamically allocated array. Refer to the information in Tables 16-5 and 16-6 if you wish to use any of the member functions.

15. STL Binary Search

Modify programming Challenge 1 in Chapter 8 (Change Account Validation) so it uses a vector instead of an array. Also, modify the program so it uses the STL `binary_search` algorithm to locate valid account numbers.



VideoNote
Solving the
Exception
Project
Problem

16. Exception Project

This assignment assumes you have completed Programming Challenge 1 of Chapter 15 (Employee and ProductionWorker Classes). Modify the Employee and ProductionWorker classes so they throw exceptions when the following errors occur:

- The Employee class should throw an exception named InvalidEmployeeNumber when it receives an employee number that is less than 0 or greater than 9999.
- The ProductionWorker class should throw an exception named InvalidShift when it receives an invalid shift.
- The ProductionWorker class should throw an exception named InvalidPayRate when it receives a negative number for the hourly pay rate.

Write a driver program that demonstrates how each of these exception conditions works.

17. Phone Book Vector

This chapter has an accompanying video note that shows how to store an object in a vector. After you view that video, write a class named PhoneBookEntry that has members for a person's name and phone number. Then write a program that creates at least five PhoneBookEntry objects and stores them in a vector. After the objects are stored in the vector, use a loop to display the contents of each object in the vector.